Reader Monad

Load+merge config.yaml, secrets.env, flags.json via Env

Season 7 · Functional · Advanced

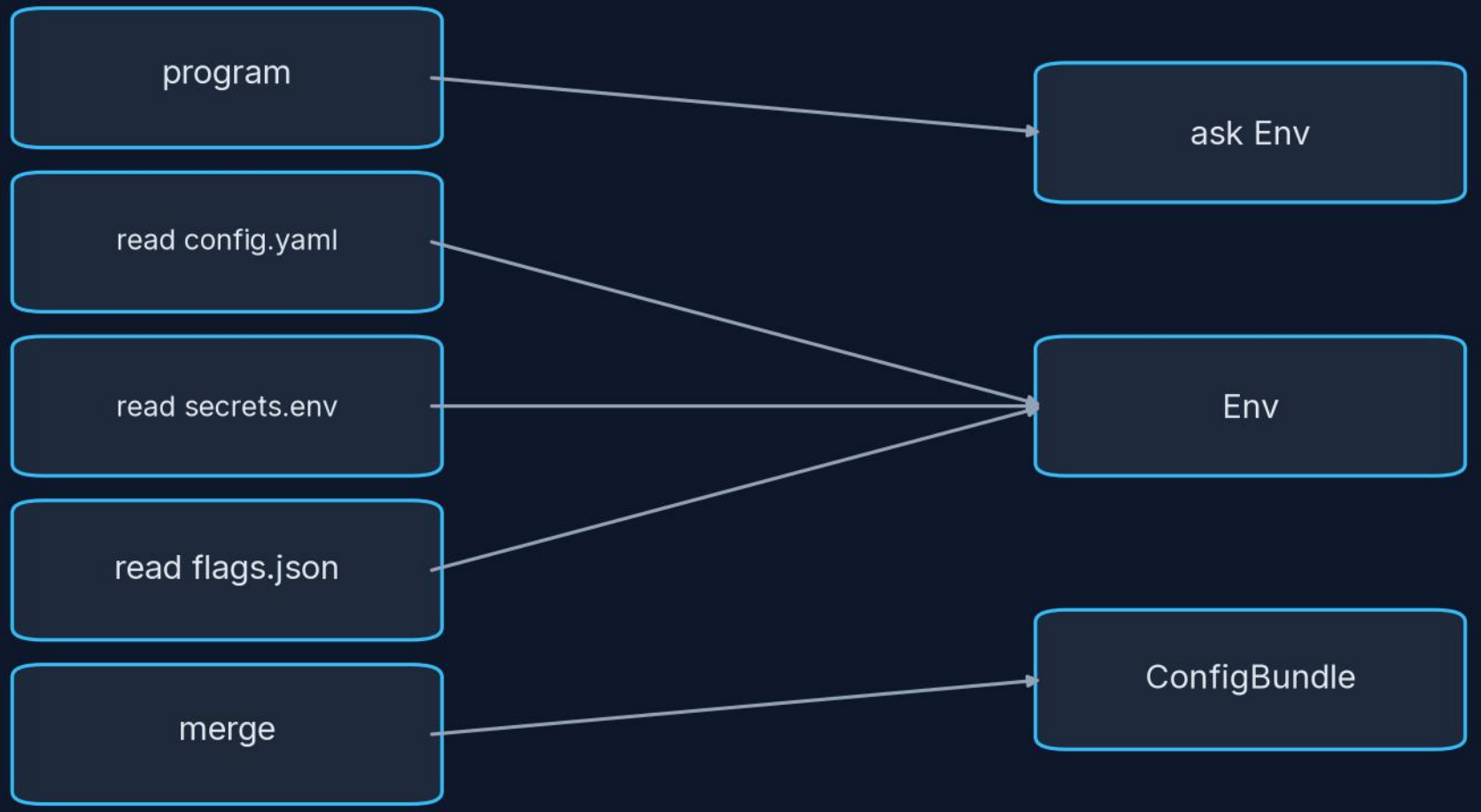
Today's Hook

- Problem: Load+merge config.yaml, secrets.env, flags.json via Env
- Naive pain cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- We will refactor to Reader with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Load config.yaml + secrets.env + feature-flags.json into ConfigBundle.
- Env holds baseDir, charset, FileSystem, logger — asked once via Reader.
- Naive drills fs/charset/baseDir into every helper (easy to mismatch).
- Success: bundle.require('app'|'TOKEN'|'beta'); env.log has 3 read lines.

Structure



Reader: Load+merge config.yaml, secrets.env, flags.json via Env

Real-World Scenario

- Running example: Load+merge config.yaml, secrets.env, flags.json via Env
- Key types: ReaderConfigLoader
- Happy-path stdout cue: ConfigBundle[yaml={app=demo, port=8080}, secrets={DB_PASS=s3cret}, flags={beta=true}]
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.functional.reader.problem;

import java.nio.charset.StandardCharsets;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Naive: every helper takes fs/charset/baseDir explicitly — parameter drilling.
 * Easy to pass the wrong charset to one of three files.
 */
public final class NaiveMultiFileLoader {
    private final FileSystem fs;
    private final String baseDir;

    public NaiveMultiFileLoader(FileSystem fs, String baseDir) {
        this.fs = fs;
        this.baseDir = baseDir;
    }

    public ConfigBundle load() {
        var yaml = parseKv(read("config.yaml"));
        var secrets = parseKv(read("secrets.env"));
        var flags = parseKv(read("feature-flags.json"));
        return new ConfigBundle(yaml, secrets, flags);
    }
```

Naive Code (2)

```
    private String read(String name) {
        // "accidentally" hard-codes UTF-8 and ignores baseDir semantics
        return fs.readString(name, StandardCharsets.UTF_8)
                .orElseThrow(() -> new IllegalStateException("missing " + baseDir + "/" + name));
    }

    static Map<String, String> parseKv(String raw) {
        Map<String, String> m = new LinkedHashMap<>();
        for (String line : raw.split("\n")) {
            line = line.trim();
            if (line.isEmpty() || line.startsWith("#") || line.startsWith("{") || line.startsWith("}")) continue;
            line = line.replace("\"", "").replace(",", "");
            int eq = line.indexOf('=');
            int colon = line.indexOf(':');
            int sep = eq >= 0 ? eq : colon;
            if (sep > 0) {
                m.put(line.substring(0, sep).trim(), line.substring(sep + 1).trim());
            }
        }
        return m;
    }
```

Naive Code (3)

```
}
```

Why Naive Hurts

- Naive: every helper takes fs/charset/baseDir explicitly — parameter drilling. Easy to pass the wrong charset to one of three files.
- Without Reader, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.reader.problem;

import com.buildwithwaqar.patterns.functional.reader.Reader;

import java.util.Map;

/**
 * Reader-based loader: Env (baseDir, charset, FileSystem, logger) is asked once
 * and threaded through composed reads — no parameter drilling.
 *
 * Nuance: Reader is not "a file reader API". It is a computation that depends on
 * an environment. Reading multiple files together is a great fit because every
 * step needs the same Env.
 */
public final class ReaderConfigLoader {
    private ReaderConfigLoader() {}

    public static Reader<Env, String> readFile(String relative) {
        return Reader.<Env>ask().flatMap(env -> Reader.pure(env.fs()
                .readString(relative, env.charset())
                .map(content -> {
                    env.info("read " + env.baseDir() + "/" + relative + " (" + content.length() + " bytes)");
                    return content;
                })
                .orElseThrow(() -> new IllegalStateException("missing " + env.baseDir() + "/" + relative))));
    }
```

Solution (2)

```
    public static Reader<Env, ConfigBundle> loadBundle() {
        return readFile("config.yaml").flatMap(yamlRaw ->
                readFile("secrets.env").flatMap(secretsRaw ->
                        readFile("feature-flags.json").map(flagsRaw -> {
                            Map<String, String> yaml = NaiveMultiFileLoader.parseKv(yamlRaw);
                            Map<String, String> secrets = NaiveMultiFileLoader.parseKv(secretsRaw);
                            Map<String, String> flags = NaiveMultiFileLoader.parseKv(flagsRaw);
                            return new ConfigBundle(yaml, secrets, flags);
                        })));
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.reader.problem;

/** Problem: load config.yaml + secrets.env + flags.json with Env injected once. */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Reader problem: multi-file config merge ===");
        var fs = new FileSystem.InMemory()
                .put("config.yaml", "app=demo\nport=8080\n")
                .put("secrets.env", "DB_PASS=s3cret\n")
                .put("feature-flags.json", "beta=true\n");

        System.out.println("--- naive (paths + charset threaded everywhere) ---");
        System.out.println(new NaiveMultiFileLoader(fs, "/app").load());

        System.out.println("--- reader (Env asked once at the edge) ---");
        var env = Env.inMemory(fs);
        ConfigBundle bundle = ReaderConfigLoader.loadBundle().run(env);
        System.out.println(bundle);
        System.out.println("log=" + env.log());
    }
}
```

Walkthrough Output

```
=== Reader problem: multi-file config merge ===
--- naive (paths + charset threaded everywhere) ---
ConfigBundle[yaml={app=demo, port=8080}, secrets={DB_PASS=s3cret}, flags={beta=true}]
--- reader (Env asked once at the edge) ---
ConfigBundle[yaml={app=demo, port=8080}, secrets={DB_PASS=s3cret}, flags={beta=true}]
log=[read /app/config/config.yaml (19 bytes), read /app/config/secrets.env (15 bytes), read /app/config/feature-flags.json (10 bytes)]
```

Step-by-Step Trace

- Match stdout to stages — look for: ConfigBundle[yaml={app=demo, port=8080}, secrets={DB_PASS=s3cret}, flags={beta=true}]
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 54-reader ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Reader
// Prefer records + sealed where they shrink the demo.
// Problem: Load+merge config.yaml, secrets.env, flags.json via Env
```

Trade-offs

- Use Reader when «Load+merge config.yaml, secrets.env, flags.json via Env» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Reader in product language.

Related Patterns

- Name the pattern usually confused with Reader and separate the forces.
- Sketch the roles for «Load+merge config.yaml, secrets.env, flags.json via Env» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Load+merge config.yaml, secrets.env, flags.json via Env
- Naive pain → Reader boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.